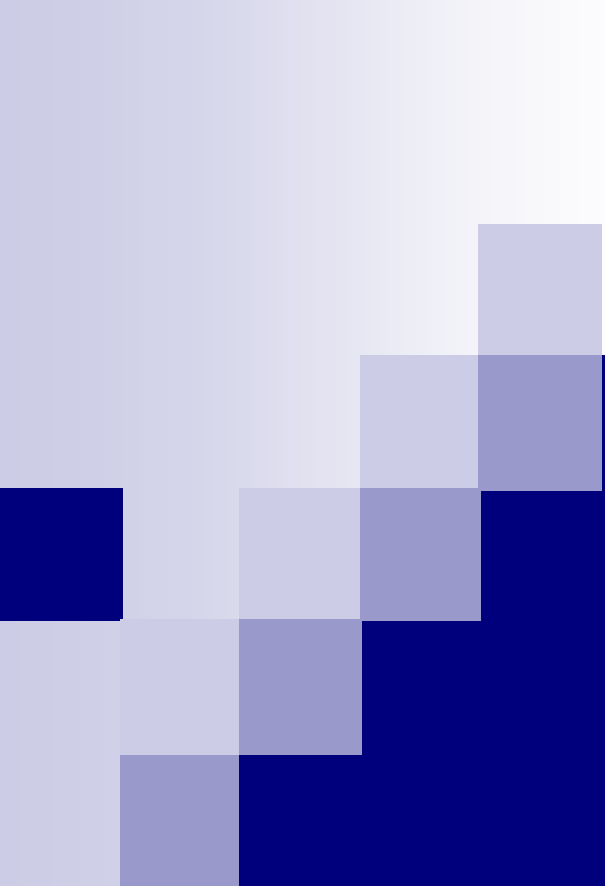




Quality Attributes of Requirements Document



Lecture # 9

Requirements Document - 1

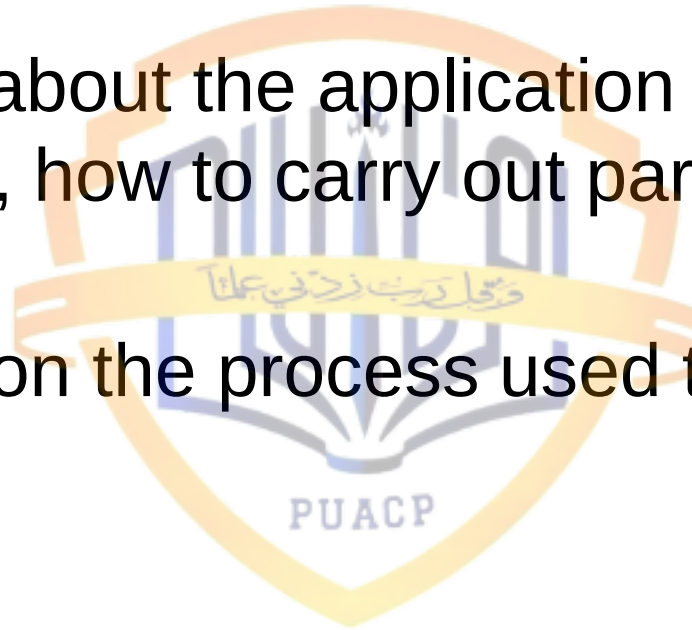
- The requirements document is a formal document used to communicate the requirements to customers, engineers and managers
- It is also known as software requirements specifications or SRS
- Requirements documents are usually written in natural languages (like, English, Japanese, French, etc.), which are ambiguous in themselves

Requirements Document - 2

- Functional requirements
- Non-functional requirements
 - Some of these are quality attributes of a software product
- Definitions of other systems which the system must integrate with

Requirements Document - 3

- Information about the application domain of the system, e.g., how to carry out particular types of computation
- Constraints on the process used to develop the system



Requirements Document - 4

- It should include both the user requirements for a system and a detailed specification of the system requirements
- In some cases, the user and system requirements may be integrated into one description, while in other cases user requirements are described before (as introduction to) system requirements

Requirements Document - 5

- For software systems, the requirements document may include a description of the hardware on which the system is to run
- The document should always include an introductory chapter which provides an overview of the system and the business needs

Requirements Document - 7

- A glossary should also be included to document technical terms
- And because multiple stakeholders will be reading documents and they need to understand meanings of different terms
- Also because stakeholders have different educational backgrounds

Requirements Document - 8

- Structure of requirements document is also very important and is developed on the basis of following information
 - Type of the system
 - Level of detail included in requirements
 - Organizational practice
 - Budget and schedule for RE process

What Should Not be Included in SRS?

- Project requirements (for example, staffing, schedules, costs, milestones, activities, phases, reporting procedures)
- Designs
- Product assurance plans (for example, configuration management plans, verification and validation plans, test plans, quality assurance plans)

Users of Requirements Documents

- System customers
- Managers
- System engineers
- System test engineers
- System maintenance engineers



Requirements for Requirements Document - 1

- It should specify only external behavior
- It should specify constraints on the implementation
- It should be easy to change
- It should serve as a reference tool for system maintainers

Requirements for Requirements Document - 2

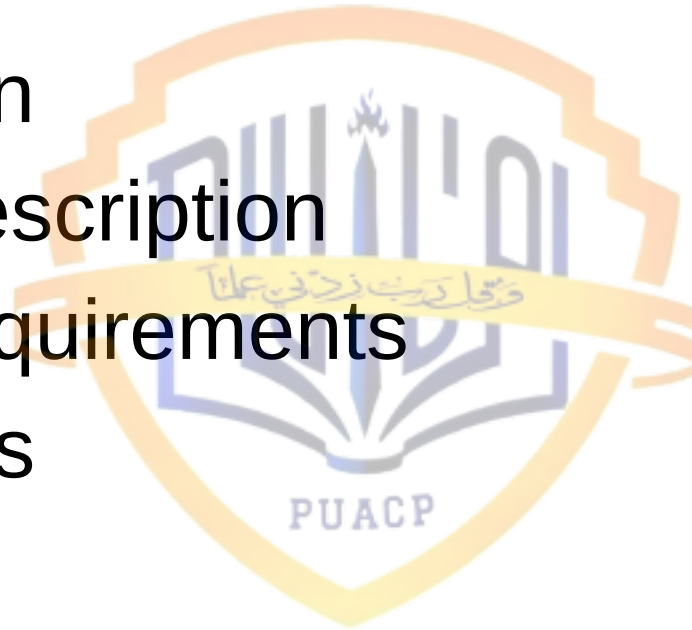
- It should record forethought about the lifecycle of the system
- It should characterize acceptable responses to undesired events
- Heninger (1980)

Organization of Requirements Document?

- Clients/developers may have their own way of organizing an SRS
- US Department of Defense
- NASA
- IEEE/ANSI 830-1993 Standard

IEEE/ANSI Standard 830-1993

1. Introduction
2. General description
3. Specific requirements
4. Appendices
5. Index



Quality Attributes of Requirements Document - 1

- Correct
- Unambiguous
- Complete
- Verifiable
- Consistent



Quality Attributes of Requirements Document - 2

- Understandable by customer
- Modifiable
- Traced
- Traceable
- Design independent



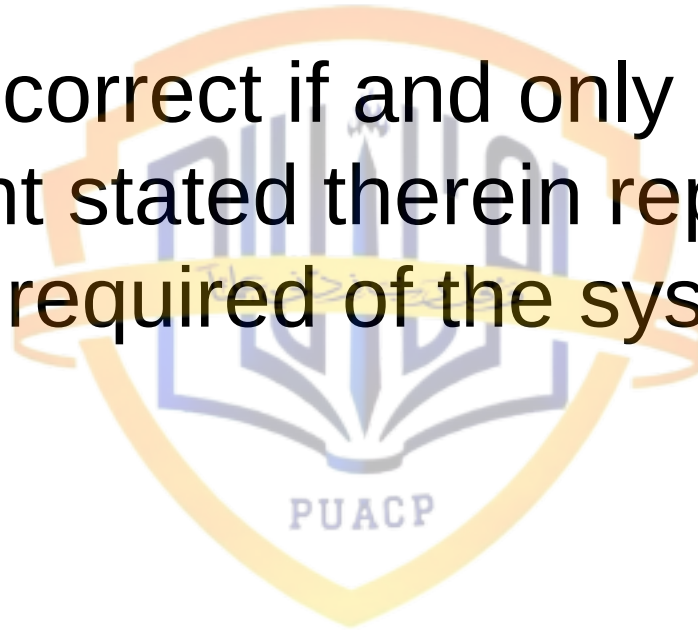
Quality Attributes of Requirements Document - 3

- Annotated
- Concise
- Organized



Correct

- An SRS is correct if and only if every requirement stated therein represents something required of the system to be built



Unambiguous

- An SRS is unambiguous if and only if every requirement stated therein has only one interpretation
- At a minimum all terms with multiple meanings must appear in a glossary
- All natural languages invite ambiguity

Example of Ambiguity

- “Aircraft that are nonfriendly and have an unknown mission or the potential to enter restricted airspace within 5 minutes shall raise an alert”
- Combination of “and” and “or” make this an ambiguous requirement

Complete - 1

- An SRS is complete if it possesses the following four qualities
 - Everything that the software is supposed to do is included in the SRS
 - Definitions of the responses of the software to all realizable classes of input data in all realizable classes of situations is included

Complete - 2

- All pages are numbered; all figures and tables are numbered, named, and referenced; all terms and units of measure are provided; and all referenced material and sections are present
- No sections are marked “To Be Determined (TBD)”

Verifiable

- An SRS is verifiable if and only if every requirement stated therein is verifiable. A requirement is verifiable if and only if there exists some finite cost effective process with which a person or machine can check that the actual as-built software product meets the requirement

Consistent - 1

- An SRS is consistent if and only if:
 - No requirement stated therein is in conflict with other preceding documents, such as specification or a statement of work
 - No subset of individual requirements stated therein conflict.

Consistent - 2

- Conflicts can be any of the following
 - Conflicting behavior
 - Conflicting terms
 - Conflicting characteristics
 - Temporal inconsistency

Understandable by Customers

- Primary readers of SRS in many cases are customers or users, who tend to be experts in an application area but are not necessarily trained in computer science

Modifiable

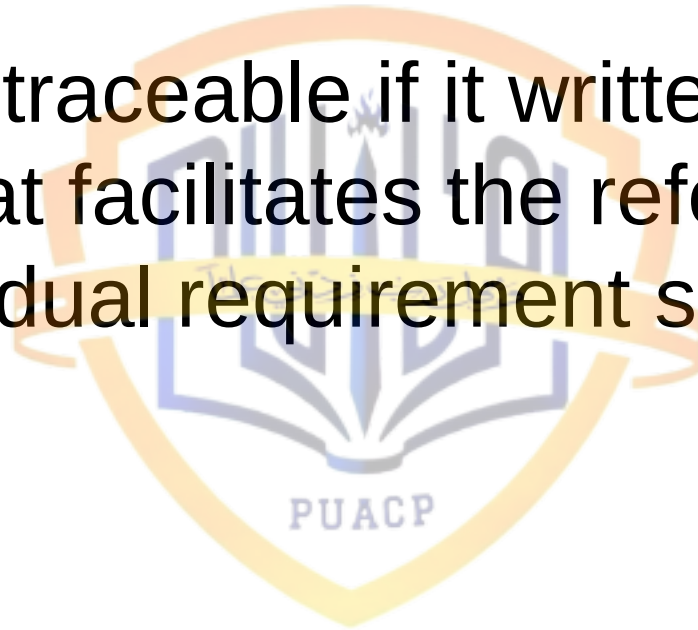
- An SRS is modifiable if its structure and style are such that any necessary changes to the requirements can be made easily, completely, and consistently
- Existence of index, table of contents, cross-referencing, and appropriate page-numbering
- This attribute deals with format and style of SRS

Traced

- An SRS is traced if the origin of its requirements is clear. That means that the SRS includes references to earlier supportive documents

Traceable

- An SRS is traceable if it is written in a manner that facilitates the referencing of each individual requirement stated therein



Techniques for Traceability

- Number every paragraph hierarchically
- Number every paragraph hierarchically and never include more than one requirement in any paragraph
- Number every requirement with a unique number in parentheses immediately after the requirement appears in the SRS
- Use a convention for indicating a requirement, e.g., use *shall* statement

Traced and Traceability - 1

- Backward-from-requirements traceability implies that we know why every requirement in the SRS exists.
- Forward-from-requirements traceability implies that we understand which components of the software satisfy each requirement

Traced and Traceability - 2

- Backward-to-requirements traceability implies that every software component explicitly references those requirements that it helps to satisfy
- Forward-to-requirements traceability implies that all documents that preceded the SRS can reference the SRS

Design Independent

- An SRS is design independent if it does not imply a specific software architecture or algorithm

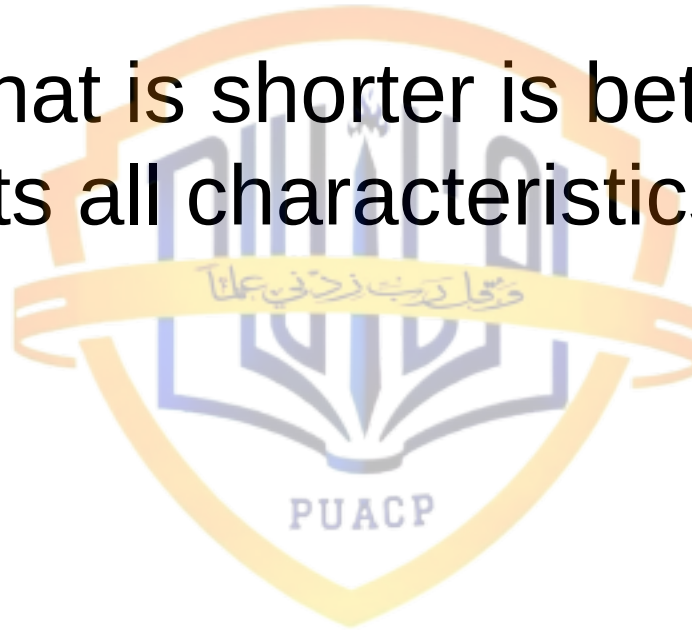


Annotated

- The purpose of annotating requirements contained in an SRS is to provide guidance to the development organization
- Relative necessity (E/D/O)
- Relative stability

Concise

- The SRS that is shorter is better, given that it meets all characteristics



Organized

- An SRS is organized if requirements contained therein are easy to locate. This implies that requirements are arranged so that requirements that are related are co-related

Phrases to Look for in an SRS - 1

- Always, Every, All, None, Never
- Certainly, Therefore, Clearly, Obviously, Evidently
- Some, Sometimes, Often, Usually, Ordinarily, Customarily, Most, Mostly
- Etc., And So Forth, And So On, Such As

Phrases to Look for in an SRS - 2

- Good, Fast, Cheap, Efficient, Small, Stable
- Handled, Processed, Rejected, Skipped, Eliminated
- If...Then...(but missing Else)

The Balancing Act

- Achieving all the preceding attributes in an SRS is impossible
- Once you become involved in writing an SRS, you will gain insight and experience necessary to do the balancing act
- There is no such thing as a perfect SRS

References

- Software Quality: Analysis and Guidelines for Success by Capers Jones
- Requirements Analysis and Specification by Alan M. Davis
- A Practitioner's Approach to Software Engineering by Roger Pressman
- Software Engineering 6th Edition, by I. Sommerville, 2000
- 'Software Engineering Quality Practices' by R. K. Kandt, Auerbach Publications, 2006